```
!pip install tensorflow scikit-learn matplotlib seaborn
```

```
Requirement already satisfied: tensorflow in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (2.20.0)
Requirement already satisfied: scikit-learn in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (1.6.1)
Requirement already satisfied: matplotlib in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (3.10.0)
Requirement already satisfied: seaborn in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (0.13.2)
Requirement already satisfied: absl-py>=1.0.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorflow
Requirement already satisfied: astunparse>=1.6.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorf
Requirement already satisfied: flatbuffers>=24.3.25 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tens
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-pack
Requirement already satisfied: google_pasta>=0.1.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensc
Requirement already satisfied: libclang>=13.0.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorfl
Requirement already satisfied: opt_einsum>=2.3.2 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorf
Requirement already satisfied: packaging in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorflow) (24
Requirement already satisfied: protobuf>=5.28.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorfl
Requirement already satisfied: requests<3,>=2.21.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensc
Requirement already satisfied: setuptools in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorflow) (7
Requirement already satisfied: six>=1.12.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorflow) (
Requirement already satisfied: termcolor>=1.1.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorfl
Requirement already satisfied: typing_extensions>=3.6.6 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from
Requirement already satisfied: wrapt>=1.11.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorflow)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensc
Requirement already satisfied: tensorboard~=2.20.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensc
Requirement already satisfied: keras>=3.10.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorflow)
Requirement already satisfied: numpy>=1.26.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorflow)
Requirement already satisfied: h5py>=3.11.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorflow)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from t
Requirement already satisfied: charset-normalizer<4,>=2 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from
Requirement already satisfied: idna<4,>=2.5 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from requests<3,>
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from reques
Requirement already satisfied: certifi>=2017.4.17 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from reques
Requirement already satisfied: markdown>=2.6.8 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorboa
Requirement already satisfied: pillow in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorboard~=2.20.
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-pa
Requirement already satisfied: werkzeug>=1.0.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from tensorboa
Requirement already satisfied: scipy>=1.6.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from scikit-learn
Requirement already satisfied: joblib>=1.2.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from scikit-lear
Requirement already satisfied: threadpoolctl>=3.1.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from scik
Requirement already satisfied: contourpy>=1.0.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplotl
Requirement already satisfied: cycler>=0.10 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplotlib)
Requirement already satisfied: fonttools>=4.22.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplot
Requirement already satisfied: kiwisolver>=1.3.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplot
Requirement already satisfied: pyparsing>=2.3.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplotl
Requirement already satisfied: python-dateutil>=2.7 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matp
Requirement already satisfied: pandas>=1.2 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from seaborn) (2.2
Requirement already satisfied: wheel<1.0,>=0.23.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from astunp
Requirement already satisfied: rich in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from keras>=3.10.0->tensc
Requirement already satisfied: namex in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from keras>=3.10.0->tens
Requirement already satisfied: optree in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from keras>=3.10.0->ten
Requirement already satisfied: pytz>=2020.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from pandas>=1.2-
Requirement already satisfied: tzdata>=2022.7 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from pandas>=1.
Requirement already satisfied: MarkupSafe>=2.1.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from werkzeu
Requirement already satisfied: markdown-it-py>=2.2.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from ric
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from r
Requirement already satisfied: mdurl~=0.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from markdown-it-py
```

```
!pip install seaborn
```

```
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from seab
Requirement already satisfied: pandas>=1.2 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from seaborn) (2.2
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from s
Requirement already satisfied: contourpy>=1.0.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplotl
Requirement already satisfied: cycler>=0.10 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplotlib!=
Requirement already satisfied: fonttools>=4.22.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplot
Requirement already satisfied: kiwisolver>=1.3.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplot
Requirement already satisfied: packaging>=20.0 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplotli
Requirement already satisfied: pillow>=8 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplotlib!=3.6
Requirement already satisfied: pyparsing>=2.3.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matplotl
Requirement already satisfied: python-dateutil>=2.7 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from matp
Requirement already satisfied: pytz>=2020.1 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from pandas>=1.2-
Requirement already satisfied: tzdata>=2022.7 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from pandas>=1.
Requirement already satisfied: six>=1.5 in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from python-dateutil>
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
```

```
import os
import time
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models

from sklearn.metrics import confusion_matrix, roc_curve, auc, precision_recall_curve, average_precision_score
```

```
DATA_DIR = './Dataset/'
IMAGE_SIZE = 224
BATCH_SIZE = 64
EPOCHS = 20
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
NUM_WORKERS = 4 if device == 'cuda' else 0
```

```
# 1. Define Data Transformations
data_transforms = {
    'Train': transforms.Compose([
        # All images are resized to the target size
        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
        # Standard augmentation for robustness
        transforms.RandomHorizontalFlip(),
        transforms.RandomRotation(10),
        # Convert to Tensor and normalize (standard for pre-trained models)
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ]),
    'Validation': transforms.Compose([
        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ]),
    'Test': transforms.Compose([
        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ]),
}

# 2. Load Datasets
print("Loading Datasets...")
image_datasets = {x: datasets.ImageFolder(os.path.join(DATA_DIR, x), data_transforms[x])
                  for x in ['Train', 'Validation', 'Test']}

# 3. Create DataLoaders (Optimized for CUDA)
dataloaders = {
    x: DataLoader(image_datasets[x],
                  batch_size=BATCH_SIZE,
                  shuffle=(x == 'Train'), # Only shuffle training data
                  num_workers=NUM_WORKERS, # Parallel data loading
                  pin_memory=(device == 'cuda')) # Fast GPU memory transfer
    for x in ['Train', 'Validation', 'Test']
}

# 4. Extract Class Information
dataset_sizes = {x: len(image_datasets[x]) for x in ['Train', 'Validation', 'Test']}
class_names = image_datasets['Train'].classes

print(f"Training Samples: {dataset_sizes['Train']}")
print(f"Validation Samples: {dataset_sizes['Validation']}")
```

```
print(f"Test Samples: {dataset_sizes['Test']}")
print(f"Class Names: {class_names}")
```

```
Loading Datasets...
Training Samples: 140002
Validation Samples: 39428
Test Samples: 10905
Class Names: ['Fake', 'Real']
```

```
def build_mobilenet_model():
    """Builds and initializes the MobileNetV3Small model."""
    print("Building MobileNetV3Small model...")

    # Load pre-trained MobileNetV3Small
    model = models.mobilenet_v3_small(weights='IMAGENET1K_V1')

    # Freeze all layers in the base model (Transfer Learning)
    for param in model.parameters():
        param.requires_grad = False

    # --- Modify the Classifier Head ---
    # The last layer is in the model.classifier[3] for mobilenet_v3_small
    num_ftrs = model.classifier[3].in_features

    # Replace the final layer for binary classification (outputting 1 logit)
    # NOTE: We use nn.Linear(..., 1) and nn.BCEWithLogitsLoss for numerical stability in PyTorch.
    model.classifier[3] = nn.Linear(num_ftrs, 1)

    # Move model to the selected device (GPU/CPU)
    model = model.to(device)

    # Define Loss Function and Optimizer
    # nn.BCEWithLogitsLoss combines Sigmoid and Binary Cross Entropy for stability.
    criterion = nn.BCEWithLogitsLoss()

    # Only optimize the parameters that are being trained (the new classifier layer)
    optimizer = optim.Adam(model.classifier.parameters(), lr=0.0001)

    return model, criterion, optimizer

model, criterion, optimizer = build_mobilenet_model()

print("\nModel Head Structure:")
print(model.classifier)
```

```
Building MobileNetV3Small model...

Model Head Structure:
Sequential(
  (0): Linear(in_features=576, out_features=1024, bias=True)
  (1): Hardswish()
  (2): Dropout(p=0.2, inplace=True)
  (3): Linear(in_features=1024, out_features=1, bias=True)
)
```

```
def train_model(model, criterion, optimizer, num_epochs=EPOCHS):
    """The main PyTorch training loop."""
    since = time.time()
    best_acc = 0.0
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}

    print("Starting model training...")

    for epoch in range(num_epochs):
        print(f'\nEpoch {epoch+1}/{num_epochs}')
        print('-' * 10)

        # Each epoch has a training and validation phase
        for phase in ['Train', 'Validation']:
            if phase == 'Train':
                model.train()  # Set model to training mode
            else:
                model.eval()   # Set model to evaluate mode

            running_loss = 0.0
            running_corrects = 0
```

```
                # Iterate over data with tqdm for progress bar
                # TQDM gives us visibility on the speed of the data pipeline.
                for inputs, labels in tqdm(dataloaders[phase], desc=f"{phase} Epoch {epoch+1}"):
                    # Move inputs and labels to the designated device (CUDA/CPU)
                    inputs = inputs.to(device)
                    labels = labels.float().unsqueeze(1).to(device) # BCEWithLogitsLoss requires Float labels

                    optimizer.zero_grad()

                    # Forward pass
                    with torch.set_grad_enabled(phase == 'Train'):
                        outputs = model(inputs)
                        loss = criterion(outputs, labels)

                        # Calculate predicted class (sigmoid is applied to logits, then thresholded at 0.5)
                        preds = (torch.sigmoid(outputs) > 0.5).long()

                        # Backward + optimize only if in training phase
                        if phase == 'Train':
                            loss.backward()
                            optimizer.step()

                    # Statistics
                    running_loss += loss.item() * inputs.size(0)
                    running_corrects += torch.sum(preds == labels.data.long())

                epoch_loss = running_loss / dataset_sizes[phase]
                epoch_acc = running_corrects.double() / dataset_sizes[phase]

                print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

                # Save history
                if phase == 'Train':
                    history['train_loss'].append(epoch_loss)
                    history['train_acc'].append(epoch_acc.item())
                else:
                    history['val_loss'].append(epoch_loss)
                    history['val_acc'].append(epoch_acc.item())

                    # Deep copy the model if it's the best validation accuracy
                    if epoch_acc > best_acc:
                        best_acc = epoch_acc
                        best_model_wts = model.state_dict()


        time_elapsed = time.time() - since
        print(f'Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s')
        print(f'Best Validation Acc: {best_acc:.4f}')

        # Load best model weights
        model.load_state_dict(best_model_wts)
        return model, history

    # Start Training
    model, history = train_model(model, criterion, optimizer, num_epochs=EPOCHS)
```

```
----------
Train Epoch 16: 100%|██████████| 2188/2188 [01:39<00:00, 22.01it/s]
Train Loss: 0.3669 Acc: 0.8333
Validation Epoch 16: 100%|██████████| 617/617 [00:27<00:00, 22.81it/s]
Validation Loss: 0.4062 Acc: 0.8147

Epoch 17/20
----------
Train Epoch 17: 100%|██████████| 2188/2188 [01:41<00:00, 21.58it/s]
Train Loss: 0.3687 Acc: 0.8321
Validation Epoch 17: 100%|██████████| 617/617 [00:26<00:00, 23.35it/s]
Validation Loss: 0.4070 Acc: 0.8144

Epoch 18/20
----------
Train Epoch 18: 100%|██████████| 2188/2188 [01:42<00:00, 21.36it/s]
Train Loss: 0.3675 Acc: 0.8339
Validation Epoch 18: 100%|██████████| 617/617 [00:28<00:00, 21.60it/s]
Validation Loss: 0.4064 Acc: 0.8146

Epoch 19/20
----------
Train Epoch 19: 100%|██████████| 2188/2188 [01:40<00:00, 21.72it/s]
Train Loss: 0.3666 Acc: 0.8341
Validation Epoch 19: 100%|██████████| 617/617 [00:27<00:00, 22.49it/s]
Validation Loss: 0.4079 Acc: 0.8144

Epoch 20/20
----------
Train Epoch 20: 100%|██████████| 2188/2188 [01:35<00:00, 22.85it/s]
Train Loss: 0.3670 Acc: 0.8335
Validation Epoch 20: 100%|██████████| 617/617 [00:25<00:00, 23.93it/s]Validation Loss: 0.4061 Acc: 0.8148
Training complete in 41m 23s
Best Validation Acc: 0.8156
```

```
def plot_training_history(history):
    """Plots the training and validation loss and accuracy curves."""

    epochs_range = range(len(history['train_acc']))

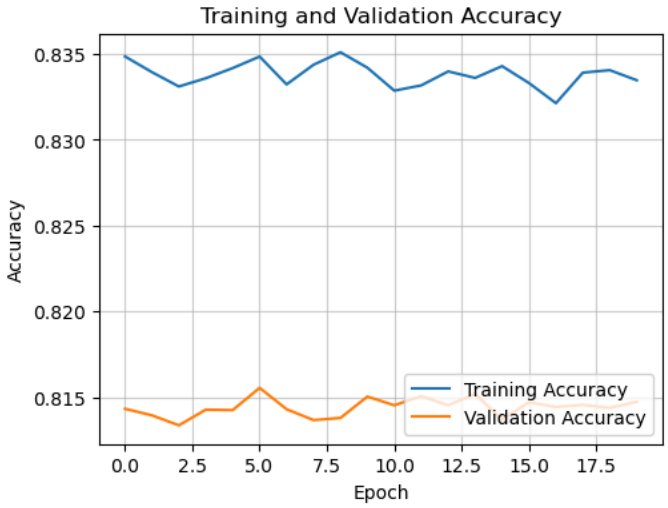
    plt.figure(figsize=(12, 4))

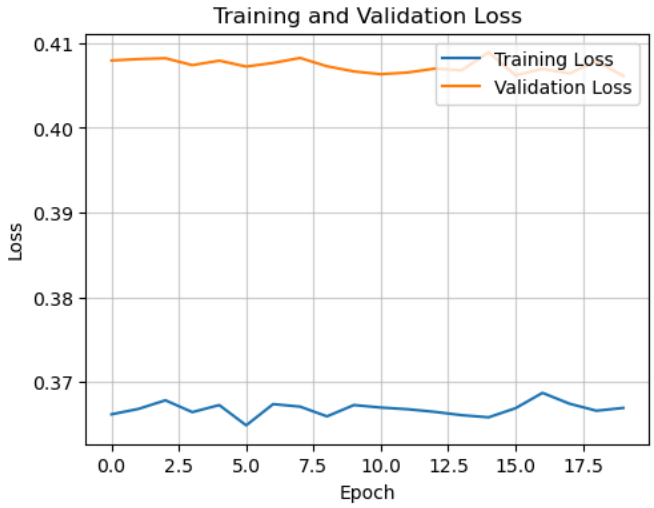
    # Accuracy Plot
    plt.subplot(1, 2, 1)
    plt.plot(epochs_range, history['train_acc'], label='Training Accuracy')
    plt.plot(epochs_range, history['val_acc'], label='Validation Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.grid(alpha=0.6)
    plt.legend(loc='lower right')

    # Loss Plot
    plt.subplot(1, 2, 2)
    plt.plot(epochs_range, history['train_loss'], label='Training Loss')
    plt.plot(epochs_range, history['val_loss'], label='Validation Loss')
    plt.title('Training and Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.grid(alpha=0.6)
    plt.legend(loc='upper right')

    plt.show()

plot_training_history(history)
```






```
def evaluate_test_set(model, dataloader):
    """Evaluates the model on the test set and returns true/predicted labels and probabilities."""
    model.eval()
    y_true = []
    y_scores = [] # Raw probabilities/scores (output of sigmoid)

    print("\nGenerating predictions for Test Set...")
    with torch.no_grad():
        for inputs, labels in tqdm(dataloader, desc="Testing"):
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)

            # Apply sigmoid to get probabilities
            probs = torch.sigmoid(outputs)

            y_true.extend(labels.cpu().numpy())
            y_scores.extend(probs.cpu().numpy())

    # Convert to NumPy arrays and flatten scores
    y_true = np.array(y_true).astype(int)
    y_scores = np.array(y_scores).flatten()

    # Convert scores to binary predictions (0 or 1)
    y_pred_classes = (y_scores > 0.5).astype(int)

    return y_true, y_pred_classes, y_scores

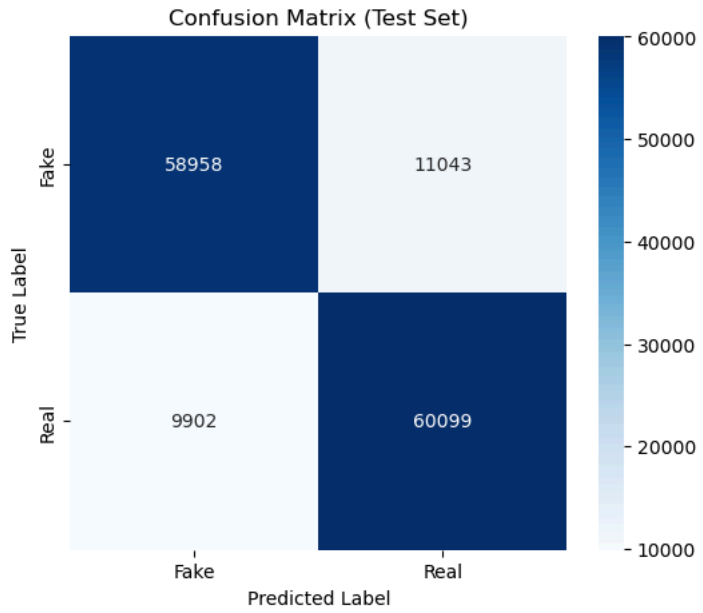
y_true, y_pred_classes, y_scores = evaluate_test_set(model, dataloaders['Test'])
```

```
Generating predictions for Test Set...
Testing: 100%|██████████████████████████| 2188/2188 [01:38<00:00, 22.27it/s]
```

```
def plot_confusion_matrix(y_true, y_pred_classes, class_labels):
    """Plots the Confusion Matrix."""
    cm = confusion_matrix(y_true, y_pred_classes)

    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_labels, yticklabels=class_labels)
    plt.title('Confusion Matrix (Test Set)')
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.show()

plot_confusion_matrix(y_true, y_pred_classes, class_names)
```




```
# 1. ROC Curve and AUC
fpr, tpr, thresholds = roc_curve(y_true, y_scores)
roc_auc = auc(fpr, tpr)

# 2. Precision-Recall Curve and AUPR
precision, recall, _ = precision_recall_curve(y_true, y_scores)
aupr = average_precision_score(y_true, y_scores)

# --- Plotting ROC and PR Curves ---
plt.figure(figsize=(12, 5))

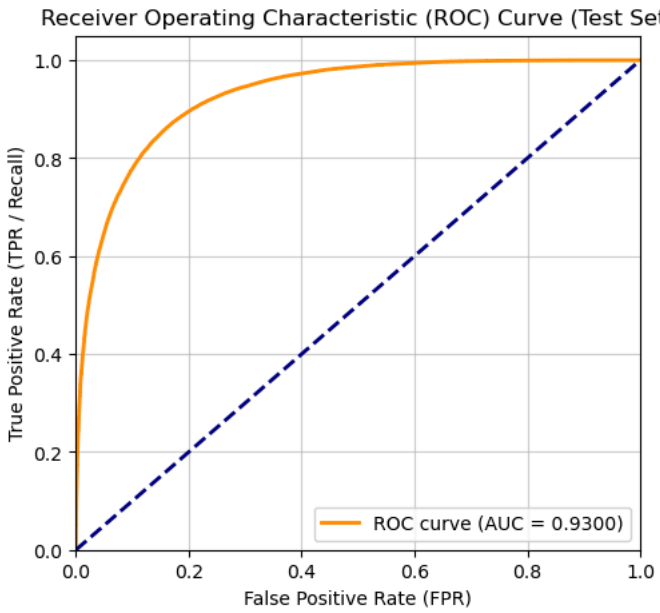
# Plot ROC Curve
plt.subplot(1, 2, 1)
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR / Recall)')
plt.title('Receiver Operating Characteristic (ROC) Curve (Test Set)')
plt.legend(loc="lower right")
plt.grid(alpha=0.6)

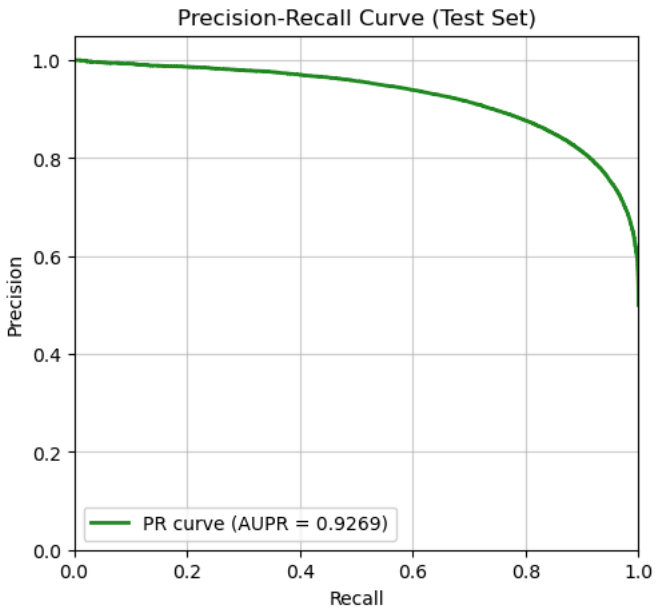
# Plot Precision-Recall Curve
plt.subplot(1, 2, 2)
plt.plot(recall, precision, color='forestgreen', lw=2, label=f'PR curve (AUPR = {aupr:.4f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve (Test Set)')
plt.legend(loc="lower left")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.grid(alpha=0.6)

plt.show()

# Final Accuracy Calculation
test_accuracy = np.mean(y_true == y_pred_classes)

print("\n--- Summary of Key Metrics ---")
print(f"Final Test Accuracy: {test_accuracy*100:.2f}%")
print(f"ROC AUC Score: {roc_auc:.4f}")
print(f"AUPR Score (Area Under PR Curve): {aupr:.4f}")
```






```
--- Summary of Key Metrics ---
Final Test Accuracy: 85.04%
ROC AUC Score: 0.9300
AUPR Score (Area Under PR Curve): 0.9269
```

```
!pip install datetime
```

```
Collecting datetime
  Downloading DateTime-5.5-py3-none-any.whl.metadata (33 kB)
Requirement already satisfied: zope.interface in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from datetime)
Requirement already satisfied: pytz in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from datetime) (2025.2)
Requirement already satisfied: setuptools in /home/aarav-shah/anaconda3/lib/python3.13/site-packages (from zope.interface
Downloading DateTime-5.5-py3-none-any.whl (52 kB)
Installing collected packages: datetime
Successfully installed datetime-5.5
```

```
# Save the final model weights
import datetime

timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
model_filename = f"deepfake_mobilenet_acc_{test_accuracy*100:.2f}_{timestamp}.pth"
torch.save(model.state_dict(), model_filename)
```